

commerce-sales-analysis-project-3

November 22, 2025

1 E-Commerce Sales Analysis Project

```
[5]: import pandas as pd
import plotly.express as px
import plotly.graph_objects as go
import plotly.io as pio
import plotly.colors as colors
pio.templates.default = "plotly_white"
```

```
[26]: pip install --upgrade plotly
```

Defaulting to user installation because normal site-packages is not writeable
Requirement already satisfied: plotly in c:\programdata\anaconda3\lib\site-packages (5.24.1)

Collecting plotly

Downloading plotly-6.5.0-py3-none-any.whl.metadata (8.5 kB)

Collecting narwhals>=1.15.1 (from plotly)

Downloading narwhals-2.12.0-py3-none-any.whl.metadata (11 kB)

Requirement already satisfied: packaging in c:\programdata\anaconda3\lib\site-packages (from plotly) (24.1)

Downloading plotly-6.5.0-py3-none-any.whl (9.9 MB)

```
----- 0.0/9.9 MB ? eta -:-:--
----- 2.1/9.9 MB 14.7 MB/s eta 0:00:01
----- 4.5/9.9 MB 12.2 MB/s eta 0:00:01
----- 5.5/9.9 MB 9.9 MB/s eta 0:00:01
----- 6.3/9.9 MB 8.8 MB/s eta 0:00:01
----- 7.3/9.9 MB 7.7 MB/s eta 0:00:01
----- 8.7/9.9 MB 7.3 MB/s eta 0:00:01
----- 9.4/9.9 MB 6.5 MB/s eta 0:00:01
----- 9.7/9.9 MB 6.2 MB/s eta 0:00:01
----- 9.9/9.9 MB 5.8 MB/s eta 0:00:00
```

Downloading narwhals-2.12.0-py3-none-any.whl (425 kB)

Installing collected packages: narwhals, plotly

Successfully installed narwhals-2.12.0 plotly-6.5.0

Note: you may need to restart the kernel to use updated packages.

WARNING: The script plotly_get_chrome.exe is installed in
'C:\Users\asus1\AppData\Roaming\Python\Python312\Scripts' which is not on PATH.

Consider adding this directory to PATH or, if you prefer to suppress this warning, use --no-warn-script-location.

```
[6]: import pandas as pd

# Read CSV file
df = pd.read_csv("Sample - Superstore.csv", encoding='latin-1')

# Optional: show first few rows
df.head()
```

```
[6]:      Row ID      Order ID  Order Date  Ship Date      Ship Mode Customer ID \
0         1  CA-2016-152156   11/8/2016  11/11/2016    Second Class    CG-12520
1         2  CA-2016-152156   11/8/2016  11/11/2016    Second Class    CG-12520
2         3  CA-2016-138688   6/12/2016   6/16/2016    Second Class    DV-13045
3         4  US-2015-108966  10/11/2015  10/18/2015    Standard Class    SO-20335
4         5  US-2015-108966  10/11/2015  10/18/2015    Standard Class    SO-20335
```

```
      Customer Name  Segment      Country      City ... \
0      Claire Gute   Consumer  United States    Henderson ...
1      Claire Gute   Consumer  United States    Henderson ...
2  Darrin Van Huff  Corporate  United States    Los Angeles ...
3  Sean O'Donnell   Consumer  United States  Fort Lauderdale ...
4  Sean O'Donnell   Consumer  United States  Fort Lauderdale ...
```

```
      Postal Code  Region      Product ID      Category Sub-Category \
0      42420      South  FUR-BO-10001798      Furniture    Bookcases
1      42420      South  FUR-CH-10000454      Furniture      Chairs
2      90036      West   OFF-LA-10000240  Office Supplies      Labels
3      33311      South  FUR-TA-10000577      Furniture      Tables
4      33311      South  OFF-ST-10000760  Office Supplies      Storage
```

```
      Product Name      Sales  Quantity \
0      Bush Somerset Collection Bookcase  261.9600      2
1  Hon Deluxe Fabric Upholstered Stacking Chairs,...  731.9400      3
2  Self-Adhesive Address Labels for Typewriters b...  14.6200      2
3      Bretford CR4500 Series Slim Rectangular Table  957.5775      5
4      Eldon Fold 'N Roll Cart System  22.3680      2
```

```
      Discount      Profit
0      0.00  41.9136
1      0.00  219.5820
2      0.00   6.8714
3      0.45 -383.0310
4      0.20   2.5164
```

[5 rows x 21 columns]

```
[7]: df
```

```
[7]:
```

	Row ID	Order ID	Order Date	Ship Date	Ship Mode	\
0	1	CA-2016-152156	11/8/2016	11/11/2016	Second Class	
1	2	CA-2016-152156	11/8/2016	11/11/2016	Second Class	
2	3	CA-2016-138688	6/12/2016	6/16/2016	Second Class	
3	4	US-2015-108966	10/11/2015	10/18/2015	Standard Class	
4	5	US-2015-108966	10/11/2015	10/18/2015	Standard Class	
...	...	...	...	...	...	
9989	9990	CA-2014-110422	1/21/2014	1/23/2014	Second Class	
9990	9991	CA-2017-121258	2/26/2017	3/3/2017	Standard Class	
9991	9992	CA-2017-121258	2/26/2017	3/3/2017	Standard Class	
9992	9993	CA-2017-121258	2/26/2017	3/3/2017	Standard Class	
9993	9994	CA-2017-119914	5/4/2017	5/9/2017	Second Class	

	Customer ID	Customer Name	Segment	Country	City	\
0	CG-12520	Claire Gute	Consumer	United States	Henderson	
1	CG-12520	Claire Gute	Consumer	United States	Henderson	
2	DV-13045	Darrin Van Huff	Corporate	United States	Los Angeles	
3	SO-20335	Sean O'Donnell	Consumer	United States	Fort Lauderdale	
4	SO-20335	Sean O'Donnell	Consumer	United States	Fort Lauderdale	
...	...	...	...	...	...	
9989	TB-21400	Tom Boeckenhauer	Consumer	United States	Miami	
9990	DB-13060	Dave Brooks	Consumer	United States	Costa Mesa	
9991	DB-13060	Dave Brooks	Consumer	United States	Costa Mesa	
9992	DB-13060	Dave Brooks	Consumer	United States	Costa Mesa	
9993	CC-12220	Chris Cortes	Consumer	United States	Westminster	

	...	Postal Code	Region	Product ID	Category	Sub-Category	\
0	...	42420	South	FUR-B0-10001798	Furniture	Bookcases	
1	...	42420	South	FUR-CH-10000454	Furniture	Chairs	
2	...	90036	West	OFF-LA-10000240	Office Supplies	Labels	
3	...	33311	South	FUR-TA-10000577	Furniture	Tables	
4	...	33311	South	OFF-ST-10000760	Office Supplies	Storage	
...	...	...	...	...	...	...	
9989	...	33180	South	FUR-FU-10001889	Furniture	Furnishings	
9990	...	92627	West	FUR-FU-10000747	Furniture	Furnishings	
9991	...	92627	West	TEC-PH-10003645	Technology	Phones	
9992	...	92627	West	OFF-PA-10004041	Office Supplies	Paper	
9993	...	92683	West	OFF-AP-10002684	Office Supplies	Appliances	

	Product Name	Sales	Quantity	\
0	Bush Somerset Collection Bookcase	261.9600	2	
1	Hon Deluxe Fabric Upholstered Stacking Chairs,...	731.9400	3	
2	Self-Adhesive Address Labels for Typewriters b...	14.6200	2	
3	Bretford CR4500 Series Slim Rectangular Table	957.5775	5	
4	Eldon Fold 'N Roll Cart System	22.3680	2	

```

...
9989          Ultra Door Pull Handle    25.2480    3
9990  Tenex B1-RE Series Chair Mats for Low Pile Car...  91.9600    2
9991          Aastra 57i VoIP phone  258.5760    2
9992  It's Hot Message Books with Stickers, 2 3/4" x 5"  29.6000    4
9993  Acco 7-Outlet Masterpiece Power Center, Wihtou...  243.1600    2

```

```

      Discount    Profit
0         0.00  41.9136
1         0.00  219.5820
2         0.00   6.8714
3         0.45 -383.0310
4         0.20   2.5164
...
9989        0.20   4.1028
9990        0.00  15.6332
9991        0.20  19.3932
9992        0.00  13.3200
9993        0.00  72.9480

```

[9994 rows x 21 columns]

2 Let's start by looking at the descriptive statistics of the dataset

```
[8]: df.describe()
```

```

[8]:
      count      Row ID  Postal Code      Sales  Quantity  Discount \
count  9994.000000  9994.000000  9994.000000  9994.000000  9994.000000
mean    4997.500000  55190.379428   229.858001    3.789574    0.156203
std    2885.163629  32063.693350   623.245101    2.225110    0.206452
min       1.000000   1040.000000    0.444000    1.000000    0.000000
25%     2499.250000  23223.000000    17.280000    2.000000    0.000000
50%     4997.500000  56430.500000    54.490000    3.000000    0.200000
75%     7495.750000  90008.000000   209.940000    5.000000    0.200000
max     9994.000000  99301.000000  22638.480000   14.000000    0.800000

```

```

      Profit
count  9994.000000
mean    28.656896
std    234.260108
min   -6599.978000
25%     1.728750
50%     8.666500
75%    29.364000
max    8399.976000

```

```
[ ]: #The dataset has an order date column. used this column to create new columns
     ↳like order month, order year, and order day, which will be very valuable for
     ↳sales and profit analysis according to time periods.
```

```
[9]: df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 9994 entries, 0 to 9993
Data columns (total 21 columns):
#   Column                Non-Null Count  Dtype
---  -
0   Row ID                9994 non-null  int64
1   Order ID              9994 non-null  object
2   Order Date            9994 non-null  object
3   Ship Date             9994 non-null  object
4   Ship Mode             9994 non-null  object
5   Customer ID           9994 non-null  object
6   Customer Name         9994 non-null  object
7   Segment               9994 non-null  object
8   Country               9994 non-null  object
9   City                  9994 non-null  object
10  State                 9994 non-null  object
11  Postal Code           9994 non-null  int64
12  Region                9994 non-null  object
13  Product ID            9994 non-null  object
14  Category              9994 non-null  object
15  Sub-Category          9994 non-null  object
16  Product Name          9994 non-null  object
17  Sales                 9994 non-null  float64
18  Quantity              9994 non-null  int64
19  Discount              9994 non-null  float64
20  Profit                9994 non-null  float64
dtypes: float64(3), int64(3), object(15)
memory usage: 1.6+ MB
```

3 Converting Date Columns

```
[10]: df['Order Date'] = pd.to_datetime(df['Order Date'])
      df['Ship Date'] = pd.to_datetime(df['Ship Date'])
      # Date Conversion: The Order Date and Ship Date columns have been converted
      ↳into datetime format for date-based analysis."
```

```
[11]: df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 9994 entries, 0 to 9993
Data columns (total 21 columns):
```

#	Column	Non-Null Count	Dtype
0	Row ID	9994 non-null	int64
1	Order ID	9994 non-null	object
2	Order Date	9994 non-null	datetime64[ns]
3	Ship Date	9994 non-null	datetime64[ns]
4	Ship Mode	9994 non-null	object
5	Customer ID	9994 non-null	object
6	Customer Name	9994 non-null	object
7	Segment	9994 non-null	object
8	Country	9994 non-null	object
9	City	9994 non-null	object
10	State	9994 non-null	object
11	Postal Code	9994 non-null	int64
12	Region	9994 non-null	object
13	Product ID	9994 non-null	object
14	Category	9994 non-null	object
15	Sub-Category	9994 non-null	object
16	Product Name	9994 non-null	object
17	Sales	9994 non-null	float64
18	Quantity	9994 non-null	int64
19	Discount	9994 non-null	float64
20	Profit	9994 non-null	float64

dtypes: datetime64[ns](2), float64(3), int64(3), object(13)

memory usage: 1.6+ MB

```
[12]: df.head()
```

```
[12]:
```

	Row ID	Order ID	Order Date	Ship Date	Ship Mode	Customer ID	\
0	1	CA-2016-152156	2016-11-08	2016-11-11	Second Class	CG-12520	
1	2	CA-2016-152156	2016-11-08	2016-11-11	Second Class	CG-12520	
2	3	CA-2016-138688	2016-06-12	2016-06-16	Second Class	DV-13045	
3	4	US-2015-108966	2015-10-11	2015-10-18	Standard Class	SO-20335	
4	5	US-2015-108966	2015-10-11	2015-10-18	Standard Class	SO-20335	

	Customer Name	Segment	Country	City	...	\
0	Claire Gute	Consumer	United States	Henderson	...	
1	Claire Gute	Consumer	United States	Henderson	...	
2	Darrin Van Huff	Corporate	United States	Los Angeles	...	
3	Sean O'Donnell	Consumer	United States	Fort Lauderdale	...	
4	Sean O'Donnell	Consumer	United States	Fort Lauderdale	...	

	Postal Code	Region	Product ID	Category	Sub-Category	\
0	42420	South	FUR-BO-10001798	Furniture	Bookcases	
1	42420	South	FUR-CH-10000454	Furniture	Chairs	
2	90036	West	OFF-LA-10000240	Office Supplies	Labels	
3	33311	South	FUR-TA-10000577	Furniture	Tables	

```

4      33311  South  OFF-ST-10000760  Office Supplies      Storage

                                Product Name      Sales  Quantity \
0      Bush Somerset Collection Bookcase  261.9600          2
1  Hon Deluxe Fabric Upholstered Stacking Chairs,...  731.9400          3
2  Self-Adhesive Address Labels for Typewriters b...  14.6200          2
3      Bretford CR4500 Series Slim Rectangular Table  957.5775          5
4      Eldon Fold 'N Roll Cart System  22.3680          2

Discount  Profit
0      0.00  41.9136
1      0.00  219.5820
2      0.00   6.8714
3      0.45 -383.0310
4      0.20   2.5164

[5 rows x 21 columns]

```

4 Adding New Date-Based Columns

```

[13]: df['Order Month'] = df['Order Date'].dt.month
      df['Order Year'] = df['Order Date'].dt.year
      df['Order Day of Week'] = df['Order Date'].dt.dayofweek

```

5 Monthly Sales Analysis

```

[29]: sales_by_month = df.groupby('Order Month')['Sales'].sum().reset_index()
      fig = px.line(sales_by_month,
                    x='Order Month',
                    y='Sales',
                    title='Monthly Sales Analysis')
      fig.show()

```



```
[ ]: #Data Grouping:

#data.groupby('Order Month')['Sales'].sum() is used to calculate the total
↳ sales for each month.

#reset_index() organizes the grouped data into a clean, structured format.

#px.line: A line chart is created to show the monthly sales trend.

#fig.show(): Displays the graph.
```

6 Sales Analysis by Category

```
[31]: sales_by_category = df.groupby('Category')['Sales'].sum().reset_index()

fig = px.pie(sales_by_category,
             values='Sales',
             names='Category',
             hole=0.5,
             color_discrete_sequence=px.colors.qualitative.Pastel)

fig.update_traces(textposition='inside', textinfo='percent+label')
fig.update_layout(title_text='Sales Analysis by Category',
                  ↳title_font=dict(size=24))

fig.show()
```

Sales Analysis by Category



```
[ ]: #groupby('Category'): Extracts category-wise sales.
```



```
#Pie Chart:

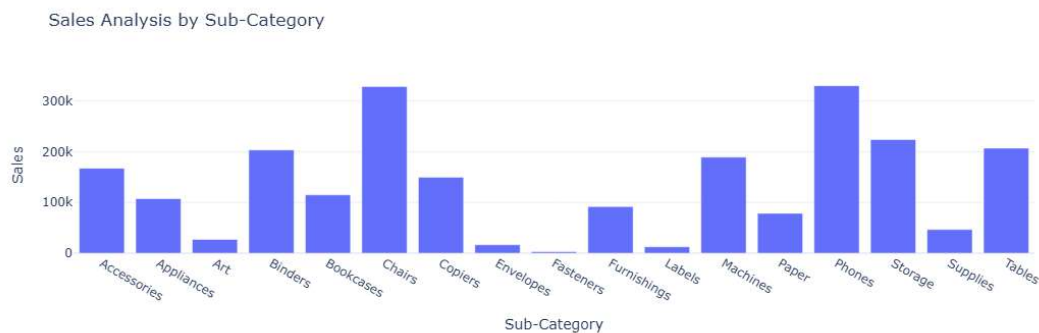
#px.pie: Displays sales proportions in a pie chart.

#hole=0.5: Creates a donut-style chart.

#Pastel Colors: A soft color palette is used in the chart.
```

7 Sales Analysis by Sub-category

```
[34]: sales_by_subcategory = df.groupby('Sub-Category')['Sales'].sum().reset_index()
fig = px.bar(sales_by_subcategory,
             x='Sub-Category',
             y='Sales',
             title='Sales Analysis by Sub-Category')
fig.show()
```



8 Monthly Profit Analysis

```
[ ]: profit_by_month = df.groupby('Order Month')['Profit'].sum().reset_index()
```

```
[27]: profit_by_month = df.groupby('Order Month')['Profit'].sum().reset_index()
fig = px.line(profit_by_month,
             x='Order Month',
             y='Profit',
             title='Monthly Profit Analysis')
fig.show()
```



9 Profit Analysis by Category

```
[19]: profit_by_category = df.groupby('Category')['Profit'].sum().reset_index()

fig = px.pie(profit_by_category,
             values='Profit',
             names='Category',
             hole=0.4,
             color_discrete_sequence=px.colors.qualitative.Pastel)

fig.update_traces(textposition='inside', textinfo='percent+label')
fig.update_layout(title_text='Profit Analysis by Category',
                  title_font=dict(size=24))

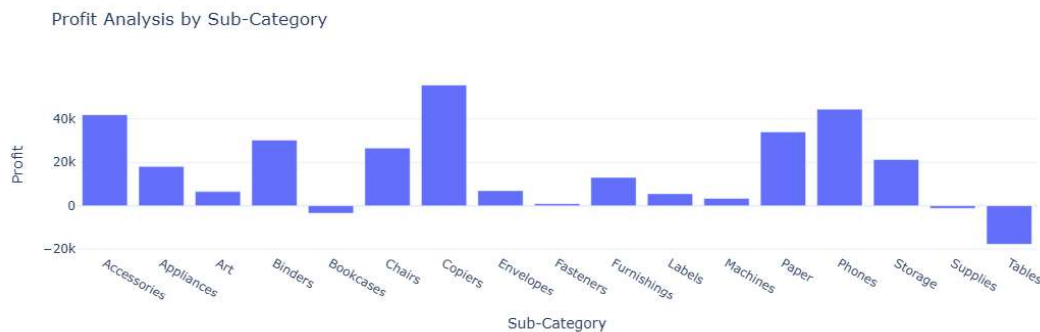
fig.show()
```

Profit Analysis by Category



10 Profit Analysis by Sub-Category

```
[21]: profit_by_subcategory = df.groupby('Sub-Category')['Profit'].sum().reset_index()
fig = px.bar(profit_by_subcategory, x='Sub-Category',
             y='Profit',
             title='Profit Analysis by Sub-Category')
fig.show()
```



11 Sales and Profit Analysis by Customer Segment

```
[23]: sales_profit_by_segment = df.groupby('Segment').agg({'Sales': 'sum', 'Profit': 'sum'}).reset_index()

color_palette = colors.qualitative.Pastel

fig = go.Figure()
fig.add_trace(go.Bar(x=sales_profit_by_segment['Segment'],
                    y=sales_profit_by_segment['Sales'],
                    name='Sales',
                    marker_color=color_palette[0]))

fig.add_trace(go.Bar(x=sales_profit_by_segment['Segment'],
                    y=sales_profit_by_segment['Profit'],
                    name='Profit',
                    marker_color=color_palette[1]))

fig.update_layout(title='Sales and Profit Analysis by Customer Segment',
                  xaxis_title='Customer Segment', yaxis_title='Amount')

fig.show()
```



12 analyse sales-to-profit ratio

```
[25]: sales_profit_by_segment = df.groupby('Segment').agg({'Sales': 'sum', 'Profit': 'sum'}).reset_index()
sales_profit_by_segment['Sales_to_Profit_Ratio'] = sales_profit_by_segment['Sales'] / sales_profit_by_segment['Profit']
print(sales_profit_by_segment[['Segment', 'Sales_to_Profit_Ratio']])
```

	Segment	Sales_to_Profit_Ratio
0	Consumer	8.659471
1	Corporate	7.677245
2	Home Office	7.125416

13 Conclusion

```
[ ]: #Optimizing high-margin products, managing low-performing categories, and
      ↪aligning strategies with seasonal trends can improve profitability and
      ↪stabilize sales.
```